James Thompkins

CS 320 Software Test, Automation

Professor Tepas

10/25/2026

CS 320 Project Two: Summary and Reflection Report

For Project One, I worked on three services: Contact, Task, and Appointment. My main goal with testing was to make sure each one followed the assignment requirements. I started by testing things that should work, like adding a valid contact, and then tested things that should not work, like entering a phone number with the wrong number of digits. I wrote JUnit tests for both the objects and their services. Across the six test files, I included 78 test methods. That gave me a lot of different situations to check, but I know the number of tests alone does not tell me the exact percentage of code covered.

For Contact, I tested the required fields, including the ID, first and last names, phone number, and address. I checked null values, fields that were too long, and phone numbers that were not exactly ten digits. I also tested adding, updating, finding, and deleting contacts. One example is testDuplicateContact in ContactServiceTest.java, which uses assertThrows(IllegalArgumentException.class, () -> service.addContact(contact2)) to make sure two contacts cannot have the same ID. This connects directly to the requirement that IDs must be unique.

For Task, I tested valid and invalid IDs, names, and descriptions. My TaskServiceTest.java also checks whether tasks can be added, updated, and deleted, and what happens when a task ID does not exist. In testUpdateTaskName, I used assertEquals to make sure the new name was actually saved. For Appointment, I focused on IDs, descriptions, and dates. I checked that an appointment cannot have a null date or a date in the past. I also tested adding and deleting appointments, duplicate IDs, and the maximum field lengths. For example, testMaximumIDLength accepts a ten-character ID, while testLongAppointmentID rejects an eleven-character one.

I feel that my tests covered the main requirements pretty well because I checked both normal situations and mistakes someone could make when using the application. I also tested values right at the limits, not just values that were obviously wrong. I did not run a separate code coverage report, so I would not say that I reached 100% coverage. To improve the project, I would run a coverage tool and look for any lines or branches I missed.

Writing JUnit tests took some practice because I had to think about how to prove the code was doing the right thing. I tried to give each test one clear purpose and use names that explained what I was checking. For example, in ContactServiceTest.java, assertEquals("2035559999", service.getContact("1001").getPhone()); checks the phone number after an update. That is stronger than just running the update and assuming it worked. In AppointmentTest.java, assertThrows(IllegalArgumentException.class, () -> new Appointment("1001", pastDate(), "Doctor appointment")); makes sure the code rejects a past appointment date. These checks helped me confirm the behavior against the requirements.

I also tried to keep my tests simple and efficient. Each service test uses a new service instance so the results of one test do not depend on another. In the appointment tests, I reused a futureDate()

helper instead of typing the same setup over and over. Its return statement, return new Date(System.currentTimeMillis() + 86400000L);, creates a date one day in the future. That saved some repeated code and made the tests easier to read. In a larger project, I would improve this by using a fixed clock so date tests are even more reliable.

The main testing technique I used was unit testing with JUnit. I tested small parts of the program on their own, which made it easier to tell where a problem came from. I used positive testing to check valid information and negative testing to make sure invalid information was rejected. I also used boundary testing, such as checking the exact ten-character ID limit and the 50-character appointment description limit. These techniques were a good fit because the project had specific rules for each field.

There are other techniques I did not use that would matter for a bigger application. Integration testing would check how these services work with other components, like a database or an API. System testing would look at the complete application instead of one class at a time. Exploratory testing would involve trying different actions and unusual inputs without following only a fixed test script. Unit tests are useful because they are quick and focused, but they cannot show whether every part of a finished application works together. I would use a mix of these approaches depending on the project.

I learned that testing requires being careful and questioning my own work. Just because I can add a contact does not mean the contact service is finished. I also need to know what happens with duplicate IDs, missing contacts, or an invalid phone number. The appointment tests were another good example because dates can cause problems if they are invalid or if the same changeable

Date object is shared between classes. Small issues like that can affect other parts of the program, so I tried to look beyond just the easiest test cases.

I also know it is easy to be biased when testing code I wrote myself. I already know what I wanted the code to do, which could make me overlook a mistake. To avoid that, I used the assignment requirements as my checklist instead of relying on what I thought should happen. I intentionally wrote tests for bad inputs and error situations. If this were a team project, I would also ask another developer to review my tests because they might notice something I missed.

This project helped me understand why software engineers need to take quality seriously. Skipping tests might make an assignment or project seem faster, but bugs can turn into bigger problems later. For example, if duplicate IDs were allowed, updating or deleting the right record could become confusing. That kind of shortcut can create technical debt because the problem takes more work to fix later. Going forward, I want to keep writing tests as I build features, run them whenever I change code, and check for missing cases before saying a project is finished. I am still learning, but this assignment made testing feel like a normal part of writing code instead of an extra step at the end.